

Programming Technique II

SECJ1023

Semester 2, 2022/2023

Faculty of Computing, UTM

Course Overview



The course covers another concept of programming:
Object-Oriented Programming (OOP)



Language used: **C++**

Course Topics

-  Overview of Programming Paradigms
-  Introduction to OOP

-  Introduction to Classes and Objects
-  Constructors and Destructors
-  Class and Object Manipulations

-  String Manipulations

-  Associations, Aggregations and Compositions
-  Inheritance
-  Polymorphisms

-  Exceptions and Templates
-  Containers and Iterators

01: Introduction to Object-oriented Programming

Programming Technique II
(SECJ1023)

*Adapted from Tony Gaddis and Barret Krupnow (2016), Starting out with
C++: From Control Structures through Objects*

Procedural Programming (PP)

 Traditional programming languages were procedural.

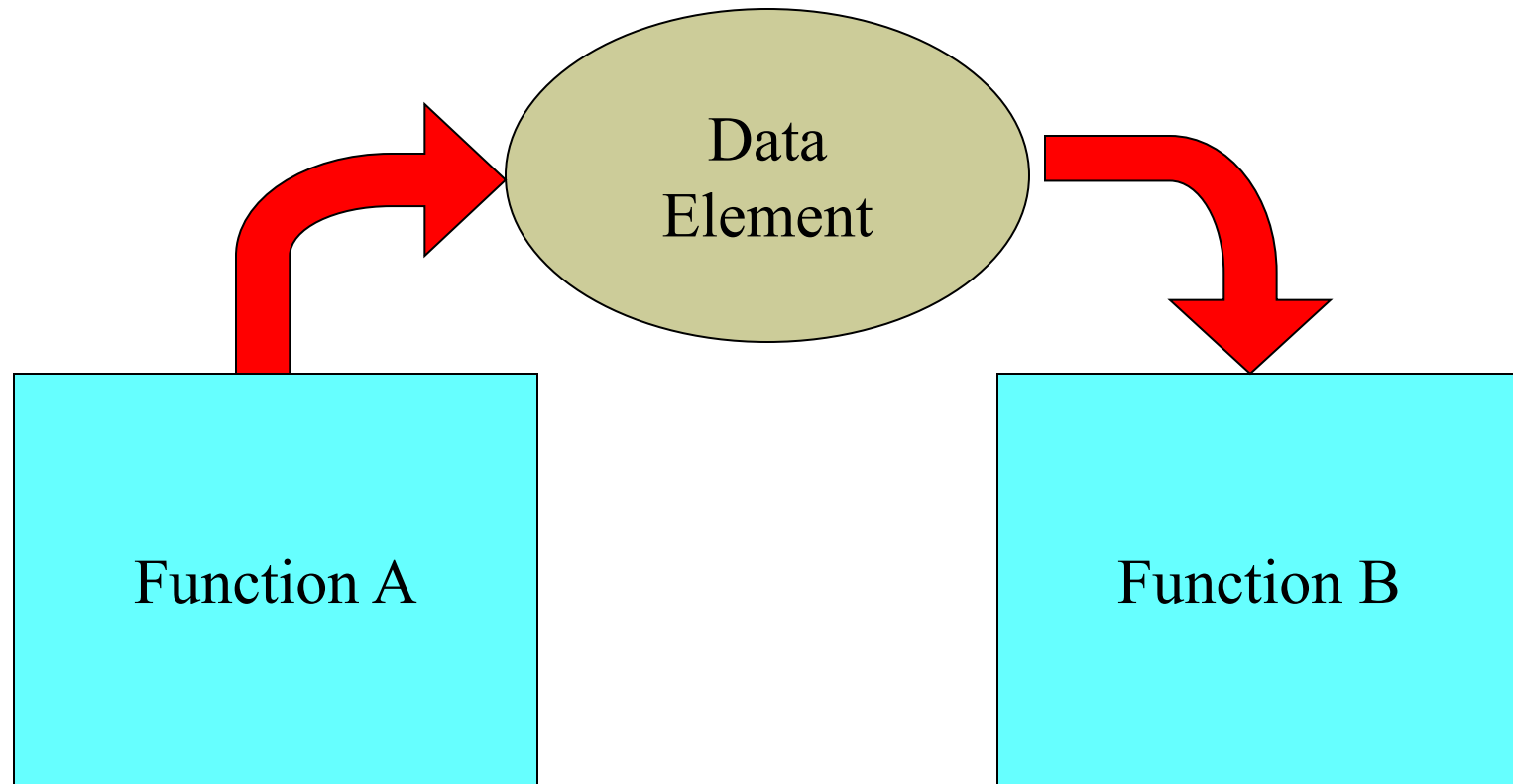
◆ C, Pascal, BASIC, Ada and COBOL

 Programming in procedural languages involves choosing data structures (appropriate ways to store data), designing algorithms, and translating algorithm into code.

 In procedural programming, **data and operations are separated**.

 This methodology requires sending data to procedure/functions

Procedural Programming



Functional Programming (FP)

-  FP is a programming paradigm where programs are constructed by applying and composing **functions**
-  Functions are treated as first-class citizen. They can be:
 - ◆ bound to names
 - ◆ passed as parameters to other functions
 - ◆ returned from other functions

-  FP uses **declarative** programming style
 - ◆ expresses the logic of **WHAT** the program should accomplish without specifying how it should achieve that.

*Note: PP and OOP use **imperative** programming style. This style focuses on describing **HOW** a program operates*

Comparison of FP to Imperative Programming

Example Problem:

Multiply all even numbers in an array by 10 and add them all, storing the final sum in the variable "result".

Both solution on the next slides are written in JavaScript, but with different programming paradigms

Traditional imperative loop

```
const numList = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
let result = 0;  
for (let i = 0; i < numList.length; i++) {  
  if (numList[i] % 2 === 0) {  
    result += numList[i] * 10;  
  }  
}
```

Functional Programming with high-order functions

```
const result = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
  .filter(n => n % 2 === 0)  
  .map(a => a * 10)  
  .reduce((a, b) => a + b);
```

FP Concepts

-  High-order and Callback Functions
-  Pure Functions
-  Lambda Functions
-  Currying
-  Recursion
-  Function composition

Further readings:

Functional programming

https://en.wikipedia.org/wiki/Functional_programming

**A Comprehensive Look at Functional Programming
(FP)**

<https://medium.com/swlh/a-comprehensive-look-at-functional-programmi>

High-order functions

- A **High-order** function (HOF) is a function that meets either one of these requirements:
 - It accepts other **functions** as **parameters**
 - It **returns a function** (*this kind of HOF is called a **factory function***)
- The **functions** that are sent as **parameters** are called **callback functions**
- Note that the sent functions will be bound rather than called to (execute)

Function Binding

A variable can hold a function

```
1  #include <iostream>
2  using namespace std;
3
4  double getNumber(){
5      return 9.5;
6  }
7  int main()
8  {
9      auto f = getNumber(); // normal function call
10     auto g = getNumber; // This is function binding, not a function call
11
12     cout << f << endl;
13     cout << g << endl;
14     cout << g() << endl;
15
16     system("pause");
17
18     return 0;
19 }
```

Output:

9.5

1

9.5

Returning Functions from a Function

In the following example, `getFunctionByOperator` is a high-order function

```
1  #include <iostream>
2  using namespace std;
3
4  double add(double a, double b){return a + b;}
5  double subtract(double a, double b){return a - b;}
6  double multiply(double a, double b){return a * b;}
7  double divide(double a, double b){return a / b;}
8
9  typedef double(BinaryFunction)(double, double);
10
11  BinaryFunction* getFunctionByOperator(char oper){
12  |  switch (oper){
13  |      case '+' : return add;
14  |      case '-' : return subtract;
15  |      case '*' : return multiply;
16  |      case '/' : return divide;
17  |  }
18  |  return NULL;
19  }
```

Return Function from Another Function

```
21  int main(){
22      auto f = getFunctionByOperator('+');
23      BinaryFunction* g = getFunctionByOperator('*');
24
25      cout << f(2,3) << endl;
26      cout << f(12,5) << endl;
27
28      cout << g(2,3) << endl;
29      cout << g(12,5) << endl;
30
31      system("pause");
32      return 0;
33 }
```

Output:

5

17

6

60

High-order Functions and Callbacks

In the following example, **add** and **multiply** are **callback functions** and **doCalculation** is a **high-order function**

```
1  #include <iostream>
2  using namespace std;
3
4  typedef double(BinaryFunction)(double, double);
5
6  void doCalculation(double a, double b, BinaryFunction f)
7  {
8      double r = f(a, b);
9      cout << "Result: " << r << endl;
10 }
11
12 double add(double a, double b){ return a + b;}
13 double multiply(double a, double b){return a * b;}
14
15 int main()
16 {
17     doCalculation(1,2, add);
18     doCalculation(5,4, multiply);
19     system("pause");
20
21     return 0;
22 }
```

Output:

Result: 3

Result: 20

To manipulate arrays

```
1  #include <iostream>
2  using namespace std;
3
4  void print(int item) { cout << item << endl; }
5
6  void printOdd(int item){
7      |   if (item % 2){ cout << item << endl; }
8      |   }
9
10 void forEach(const int list[], int size, void (*f)(int)){
11     |   for (int i = 0; i < size; i++)
12     |       f(list[i]);
13     |   }
14
```

```
15  ✓ int main(){
16      int numbers[] = {1, 2, 5, 6, 3};
17
18      forEach(numbers, 5, print); // Print all numbers in the array
19      cout << endl;
20
21      forEach(numbers, 5, printOdd); // Print odd numbers
22      cout << endl;
23
24      // Print numbers greater than 3
25      forEach(numbers, 5, [](int item){ if (item>3) cout << item << endl; } );
26
27      system("pause");
28      return 0;
29 }
```

Lambda Functions

- Lambda functions are functions without names.
- They can be written inline (no need declaration)
- They cannot be re-used (i.e. they execute only once)
- Some use cases of Lambda functions:
 - Function binding
 - Passing functions as parameters to another function.
 - Returning a function from another.

Lambda Function Example 1: Function Binding

- You can assign a variable with a function directly or inline (i.e., no need to declare the function first)

```
1
2
3 #include <iostream>
4 #include <string>
5 using namespace std;
6
7 int main()
8 {
9     auto plus = [](auto a, auto b){ return a + b;};
10
11     cout << "Integer plus: " << plus(20,1) << endl;
12     cout << "Number plus : " << plus(2.5,1.3) << endl;
13     cout << "String plus : " << plus( string("Hello "), string("World") )<< endl;
14
15     cin.get();
16     return 0;
17 }
```

Lambda function

Output:

Integer plus: 21
Number plus: 3.8
String plus: Hello World

Lambda Function Example 2: Passing functions as parameters

- A callback can be directly written to the high-order function (without having it defined first).
- This is called **Lambda function** (or Anonymous function, i.e. nameless or no name function)

```
1  #include <iostream>
2  using namespace std;
3
4  typedef double(BinaryFunction)(double, double);
5
6  void doCalculation(double a, double b, BinaryFunction f)
7  {
8      double r = f(a, b);
9      cout << "Result: " << r << endl;
10 }
11
12 int main()
13 {
14     doCalculation(1,2, [](double a, double b){ return a + b; } );
15     doCalculation(5,4, [](double a, double b){ return a * b; } );
16     system("pause");
17
18     return 0;
19 }
```

Lambda function

Lambda function

Output:

Result: 3

Result: 20

Lambda Function Example 3: Returning a function from another function

- In the following example, **powerOf** is a **function factory** (a function that returns another function).
- square**, **cube** and **squareRoot** are functions generated from the function factory.

```
3  #include <iostream>
4  #include <cmath>
5
6  using namespace std;
7
8  auto powerOf(float exponent)
9  {
10     return [exponent](float number) { return pow(number, exponent); };
11 }
12
13 int main()
14 {
15     auto square = powerOf(2);
16     auto cube = powerOf(3);
17     auto squareRoot = powerOf(0.5);
18
19     cout << "Square: " << square(25) << endl;
20     cout << "Cube: " << cube(2) << endl;
21     cout << "Square Root: " << squareRoot(2) << endl;
22
23     cin.get();
24     return 0;
25 }
26
```

Lambda function

Output:

Square: 625

Cube: 8

Square Root: 1.41421

Currying

- **Currying** is a technique from Functional Programming that transforms a **function accepting multiple parameters** into **several functions accepting single parameter**
- The translation looks something like this:

```
originalFunction(param1, param2, param3)  
curriedFunction(param1)(param2)(param3)
```

- Currying is implemented by wrapping a function inside a function, i.e., the function returns the other function.
- The parent function takes the first parameter and returns the function that takes the next parameter and this repeats until the last parameter.

Currying

- From the previous example, there will be three function calls at the **curriedFunction**:
 - Call to the parent function, i.e., **curriedFunction** which takes `param1` and returns another function (we call this function, **a**).
a = curriedFunction(param1)
 - Call to the function **a** which takes `param2` and returns another function (we call this function, **b**).
b = a(param2)
 - Call to the function **b** which takes `param3` and returns the result.
result = b(param3)


```
2
3  #include <iostream>
4  using namespace std;
5
6  // Normal function to calculate volume of a cuboid
7  int volume(int width, int length, int height)
8  {
9      return width * length * height;
10 }
11
12 // Curried version of the above function
13 auto curriedVolume(int width)
14 {
15     return [width](int length) {
16         return [width, length](int height){
17             return width * length * height;
18         };
19     };
20 }
21
22 int main()
23 {
24     cout << "Volume1: " << volume(2,5,10) << endl;
25
26     cout << "Volume2: " << curriedVolume(2)(5)(10) << endl;
27
28     // Call the curried function step-by-step
29     auto a = curriedVolume(2); // a is a generated function
30     auto b = a(5); // b is a generated function
31     cout << "Volume3: " << b(10) << endl;
32
33     cin.get();
34     return 0;
35 }
```

Output:

Volume1 : 100
Volume2 : 100
Volume3 : 100

Recursion

- Recursion is a technique that uses functions to call to themselves.
- Such functions are called recursive functions.
- A recursion instruction continues until another instruction prevents it.
- Iterations in PP paradigm are done with loops. In FP paradigm, mostly using recursion to perform repeating tasks.
- A recursion code normally consists two parts:
 - Recursion. Parts where the function calls to itself
 - Base case (or termination condition). To end the recursion.

```
6
7  #include <iostream>
8  using namespace std;
9
10 // Function to sum up all numbers from m to n using regular loop
11 int sumNumbers(int m, int n)
12 {
13     int sum = 0;
14     for (int i = m; i ≤ n; i++)
15         sum = sum + i;
16     return sum;
17 }
18
19 // The recursion version of the above function
20 int recursiveSumNumbers(int m, int n)
21 {
22     if (m == n)
23         return m;
24     return m + recursiveSumNumbers(m + 1, n);
25 }
26
27 int main()
28 {
29     cout << "Sum1: " << sumNumbers(1,5) << endl;
30     cout << "Sum2: " << recursiveSumNumbers(1,5) << endl;
31
32     cin.get();
33     return 0;
34 }
```

Base case

Recursion case

Output:

Sum1: 15

Sum2: 15

Object-Oriented Programming (OOP)



OOP is centred on **objects** rather than procedures / functions.



Objects are a melding of **data** and **operations** that manipulate that data.

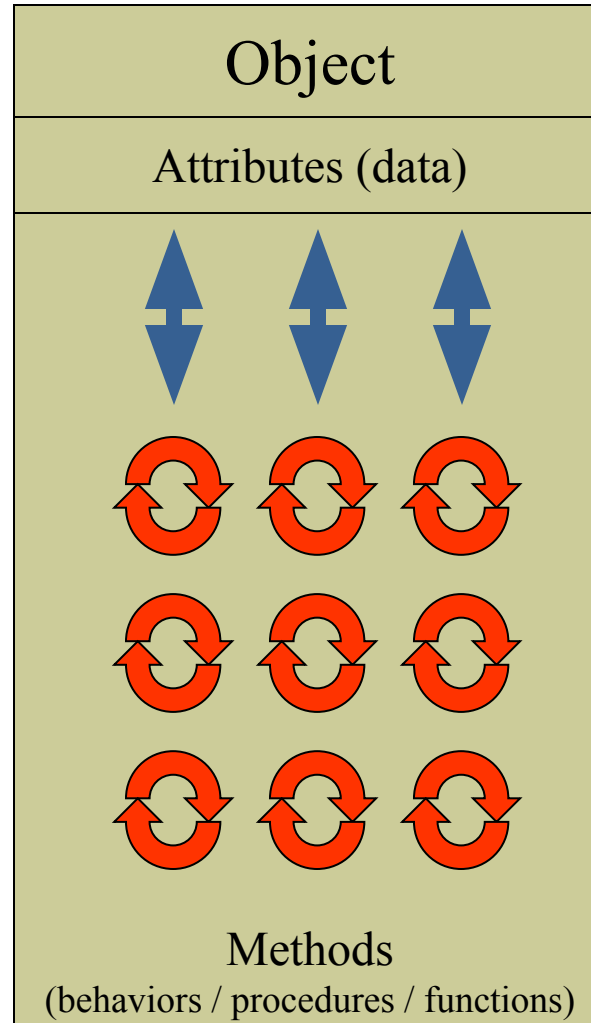


Data in an object are known as **properties** or **attributes** .



Operations/**functions** in an object are known as **methods**.

Object-Oriented Programming



Object-Oriented Programming



Object-oriented programming **combines data and methods** via **encapsulation**.



Data hiding is the ability of an object to **hide data from other objects** in the program



Only object's methods should be able to directly manipulate its attributes

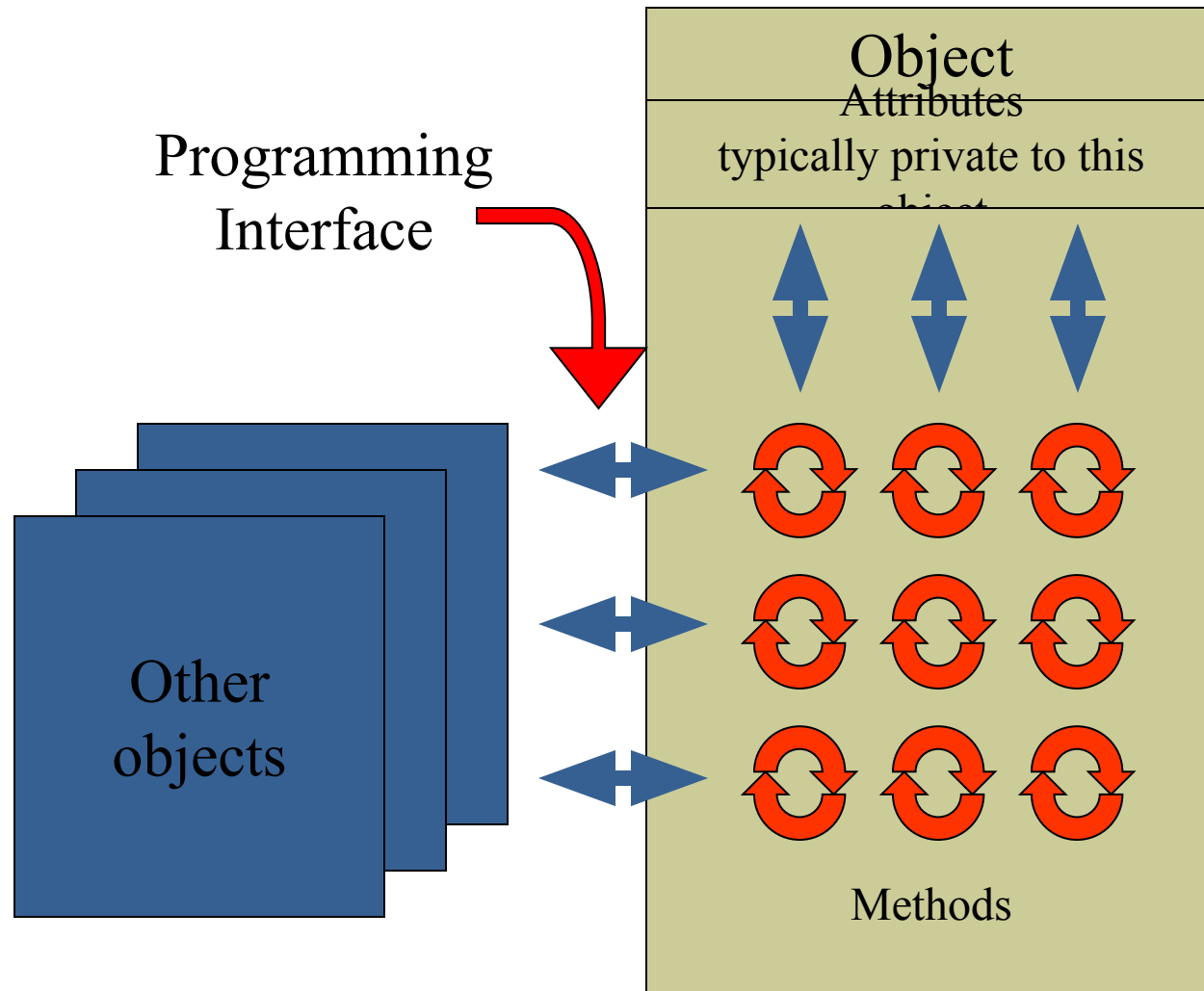


Other objects are allowed to manipulate object's attributes via the object's methods.



This indirect access is known as a **programming interface**

Object-Oriented Programming



Object-Oriented Programming



Object-oriented programming combines data and methods via **encapsulation**.



Data hiding is the ability of an object to hide data from other objects in the program



Only object's methods should be able to directly manipulate its attributes



Other objects are allowed to manipulate object's attributes via the object's methods.



This indirect access is known as a **programming interface**

OOP Principles: **Classes**



A class is the **template** or **mould** or **blueprint** from which objects are actually made.



A class **encapsulates** the attributes and actions that characterizes a certain type of object.

OOP Principles: **Objects**

-  Classes can be used to **instantiate as many objects** as are needed.
-  Each object that is created from a class is called an **instance of the class**.
-  A **program** is simply a **collection of objects** that **interact with each other** to accomplish a goal.

Classes and Objects

The ***Car* class** defines the attributes and methods that will exist in all objects that are instances of the class.

Car class

**Kancil
object**

The Kancil object is an instance of the Car class.

The Nazaria object is an instance of the Car class.

**Nazaria
object**

OOP Principles: **Encapsulation**



Encapsulation is a key concept in working with objects:
Combining attributes and methods in one package and
hiding the implementation of the data from the user of the
object.

Encapsulation:

Attributes/data
+
Methods/functions = Class

Example:

a car has attributes and methods below.

Car
<i>Attributes:</i> model, cylinder capacity
<i>Methods:</i> move, accelerate

OOP Principles: **Data Hiding**

 **Data hiding** ensures methods **should not directly access** instance attributes in a class other than their own.

 Programs should interact with object attributes only through the object's methods.

-  Data hiding is important for several reasons.
- ◆ It **protects attributes** from **accidental corruption** by **outside objects**.
 - ◆ It **hides** the **details** of **how an object works**, so the programmer can concentrate on using it.
 - ◆ It allows the maintainer of the object to have the ability to **modify** the internal functioning of the **object without “breaking” someone else's code**.

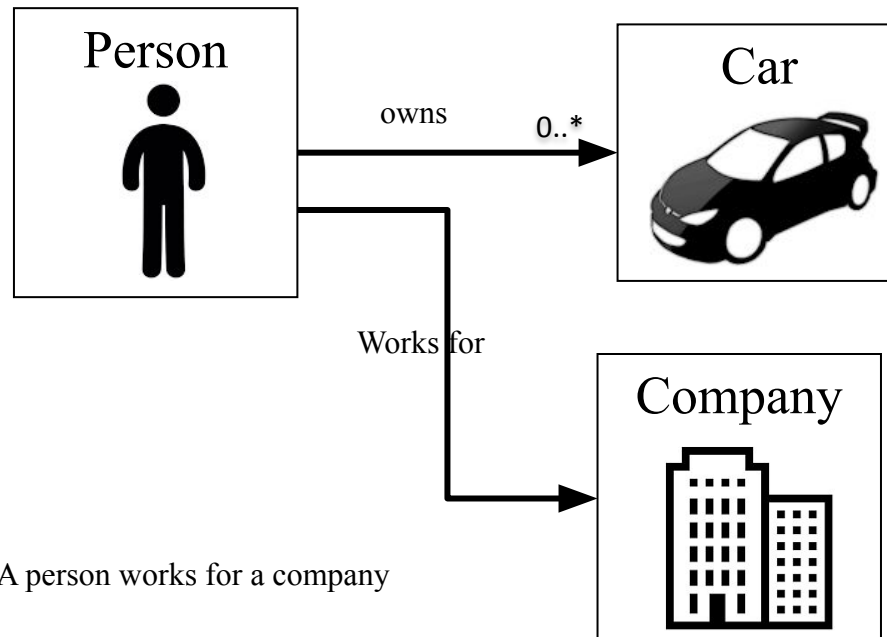
OOP Principles: Associations

 **Association:** relates classes to each other through their objects.

 Association can be, one to one, one to many, many to one, or many to many relationships.

Example:

A person can own several cars



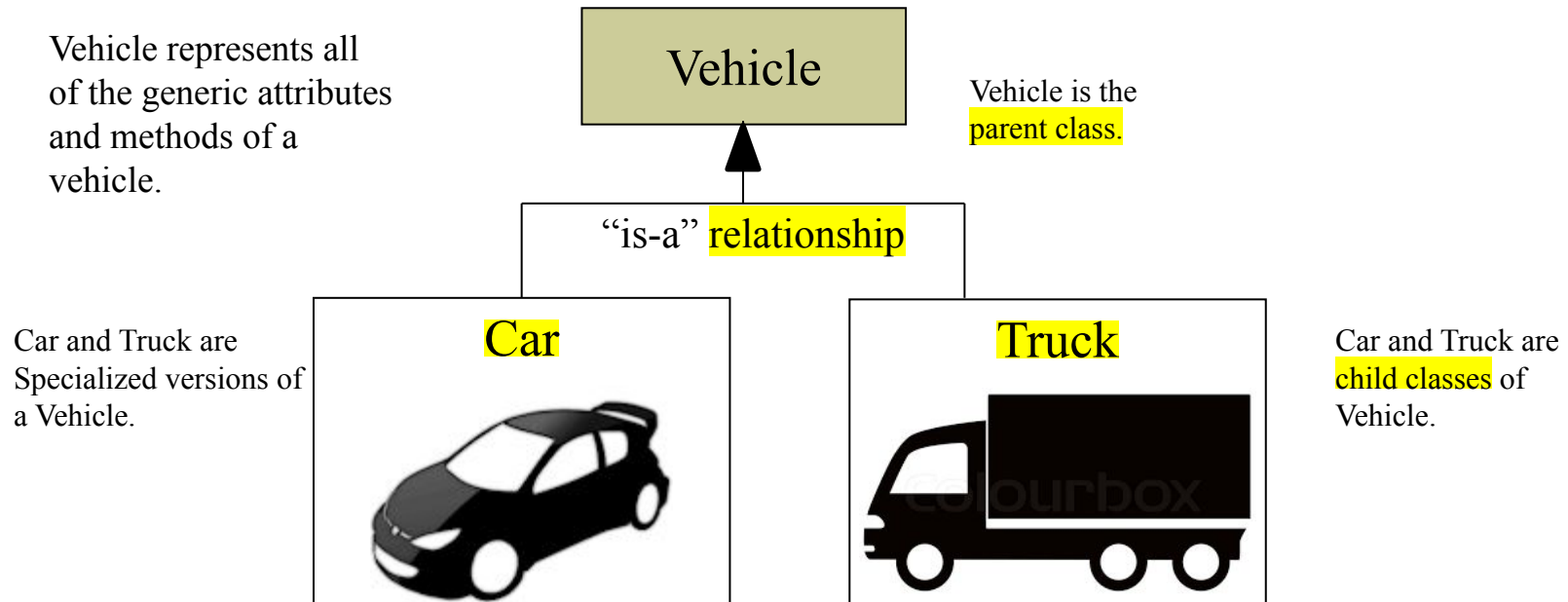
A person works for a company

OOP Principles: Inheritance

 Inheritance is the ability of **one class** to **extend** the **capabilities** of another.

- ◆ it allows **code defined in one class** to be **reused in other classes**

Example:



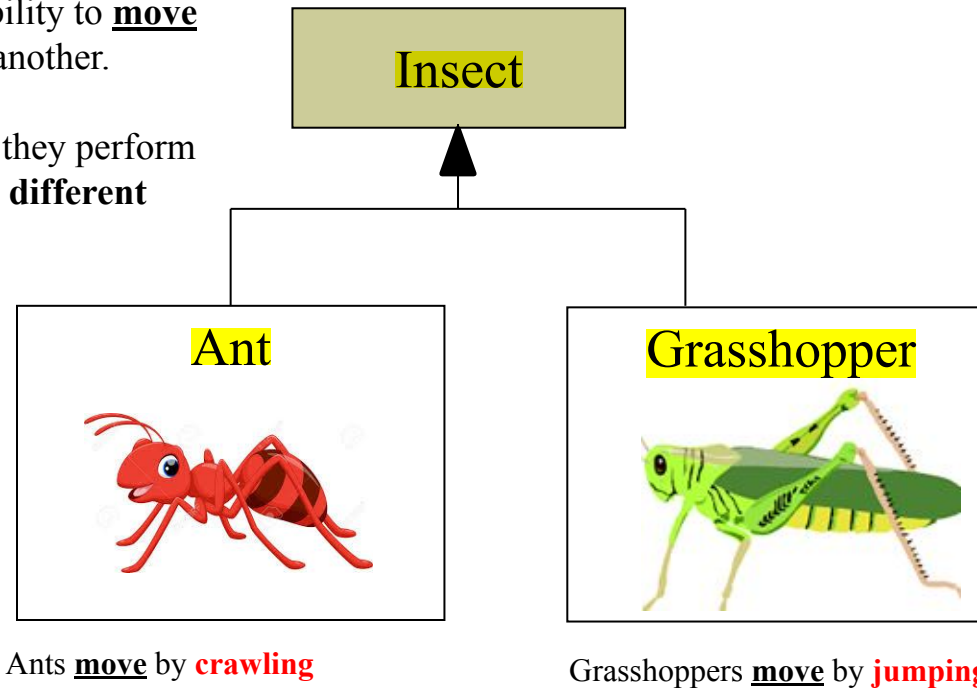
OOP Principles: Polymorphism

 Polymorphism is the ability of objects **performing the same actions differently**.

Example:

Insects have the ability to move from one point to another.

However, the way they perform their **movement is different**



Self-test: Introduction to Object Oriented Programming

-  State the differences between procedural programming and Object Oriented Programming.
-  What is an Object and what is a Class? What is the difference between them?
-  What is an Attribute?
-  What is a Method?
-  What is encapsulation? How it relates to data hiding?
-  What is association?
-  What is inheritance? How it relates to polymorphism?

The Unified Modeling Language

Programming Technique II
(SCSJ1023)

*Adapted from Tony Gaddis and Barret Krupnow (2016), Starting out with
C++: From Control Structures through Objects*

The Unified Modelling Language



UML stands for Unified Modelling Language.

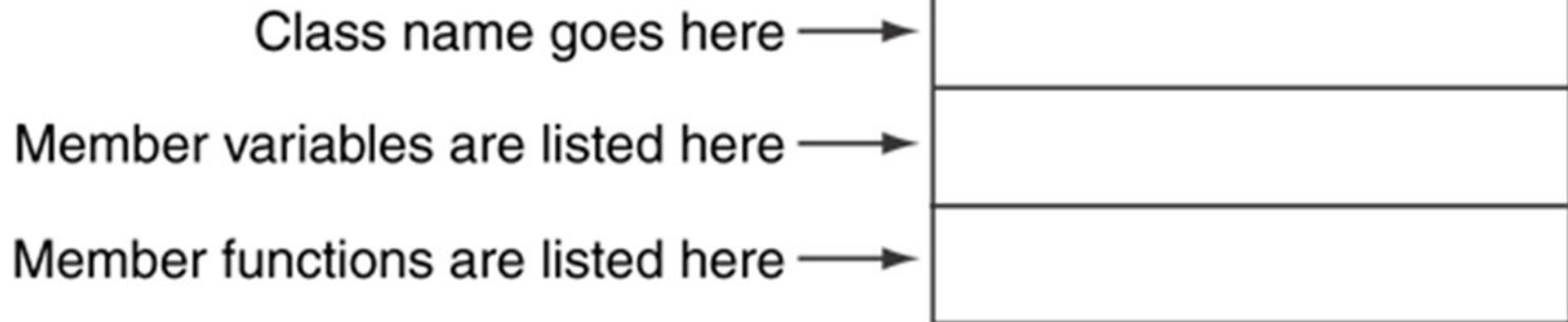


The UML provides a set of standard diagrams for graphically depicting object-oriented systems

UML Class Diagram



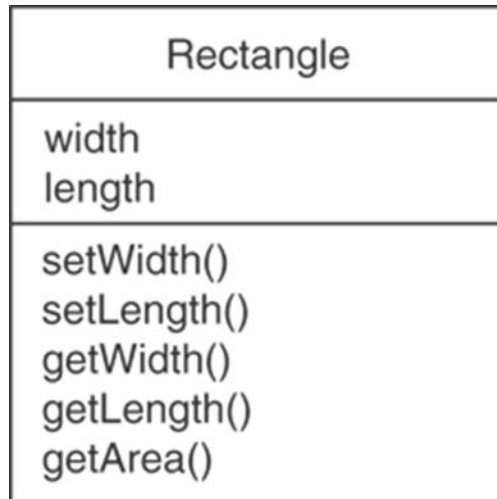
A UML diagram for a class has three main sections.



Example: A Rectangle Class



A UML diagram for a class has three main sections.



```
class Rectangle
```

```
{
```

```
    private:
```

```
        double width;
```

```
        double length;
```

```
    public:
```

```
        bool setWidth(double) ;
```

```
        bool setLength(double) ;
```

```
        double getWidth() const;
```

```
        double getLength() const;
```

```
        double getArea() const;
```

```
};
```

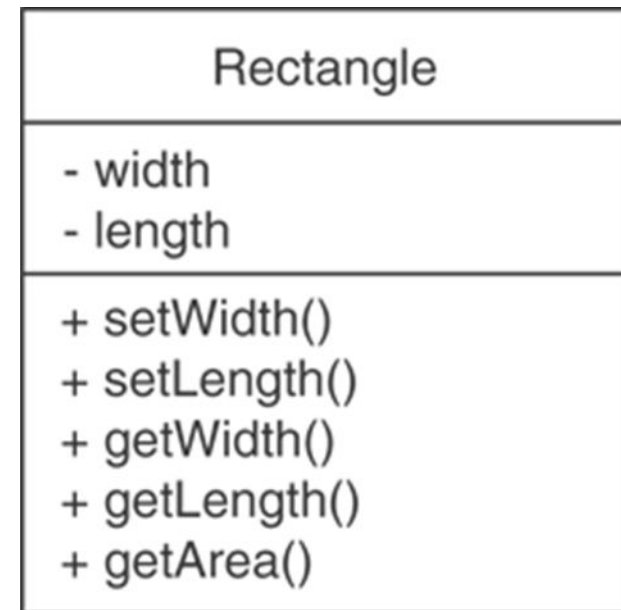
UML Access Specification Notation



In UML you indicate a private member with a minus (-) and a public member with a plus(+).

These member variables are private.

These member functions are public.



UML Data Type Notation



To indicate the data type of a member variable, place a colon followed by the name of the data type after the name of the variable.



- width : double
- length : double

UML Parameter Type Notation



To indicate the data type of a function's parameter variable, place a colon followed by the name of the data type after the name of the variable.



+ setwidth(w : double)

UML Function Return Type Notation



To indicate the data type of a function's return value, place a colon followed by the name of the data type after the function's parameter list.



+ setwidth(w : double) : void

The Rectangle Class

Rectangle
- width : double - length : double
+ setWidth(w : double) : bool + setLength(len : double) : bool + getWidth() : double + getLength() : double + getArea() : double

Showing Constructors and Destructors

No return type listed for constructors or destructors

Constructors

Destructor

InventoryItem
- description : char* - cost : double - units : int - createDescription(size : int, value : char*) : void
+ InventoryItem() : + InventoryItem(desc : char*) : + InventoryItem(desc : char*, c : double, u : int) : + ~InventoryItem() : + setDescription(d : char*) : void + setCost(c : double) : void + setUnits(u : int) : void + getDescription() : char* + getCost() : double + getUnits() : int